

Reviewer Pack — Minimal Rank of Local Ring Extensions over Boolean Algebra v...

Entry 3be5f7fa76e2 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-24 09:00:25 UTC

Minimal Rank of Local Ring Extensions over Boolean Algebra vs AC0 Parity Circuit Size

Entry ID: 3be5f7fa76e2 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-24 09:00:25 UTC

1. Conjecture

Field A (mathematical branch): Local Ring Theory **Field B** (complexity object): Complexity Theory (AC0 Parity Complexity)

Statement:

[‘For any Boolean function f represented by an AC0 parity circuit C , there exists a local ring extension L over the Boolean algebra $\{0,1\}$ such that the minimal rank of L is at most $\log_2(\text{size}(C))$.’, ‘Moreover, for any instance of size $n \leq 40$ with property P , if the minimal rank of the corresponding local ring extension is greater than $\log_2(n)$, then f cannot be computed by an AC0 parity circuit.’]

Rationale (proposer’s reasoning):

[‘Local ring theory has been applied to algebraic complexity theory before, but primarily in the context of polynomial-time operations. By focusing on local ring extensions over Boolean algebras, we can explore a less-studied aspect of this connection.’, ‘The conjecture aims to establish a bridge between the algebraic structure of local rings and the combinatorial properties of AC0 parity circuits, potentially revealing new insights into circuit complexity.’]

Taxonomy category: AC0_PARITY (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 14a5e065019af3e3

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all Boolean functions f with size $n \leq 40$, the minimal rank of the corresponding local ring extension L is at most $\log_2(\text{size}(C))$ AND no instance has a $\text{minRank}(L) > \log_2(n)$. Falsified if there exists an instance with $\text{minRank}(L) > \log_2(n)$ for any size $n \leq 40$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - (local ring theory) AND (AC0 parity complexity) AND (minimal rank) - (Boolean algebra) AND (AC0 circuit size) AND (ring extension rank) - (ring extensions over Boolean algebra) AND (AC0 circuits) AND (min rank)

Top relevant hits considered: - [http://arxiv.org/abs/1102.2932v2] Applications of Monotone Rank to Complexity Theory - [http://arxiv.org/abs/1403.8106v1] Recent advances on the log-rank conjecture in communication complexity - [http://arxiv.org/abs/2211.06357v2] Parity of ranks of Jacobians of curves - [http://arxiv.org/abs/2504.14101v2] A streamlined demonstration that stabilizer circuits simulation reduces to Boolean linear algebra - [http://arxiv.org/abs/1806.02767v3] Free Extensions and Jordan type - [http://arxiv.org/abs/2411.02936v3] Conditional Complexity Hardness: Monotone Circuit Size, Matrix Rigidity, and Tensor Rank - [http://arxiv.org/abs/1406.3065v2] Lower Bounds for Tropical Circuits and Dynamic Programs - [http://arxiv.org/abs/2207.08701v3] Notes on Boolean Read-k and Multilinear Circuits

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=124, elapsed=240.3s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def ac0_parity_circuit_size(f):
        n = len(f)
        if n == 1:
            return 1
        size = 1
        while n > 1:
            n //= 2
            size *= 2
        return size

    def local_ring_extension_rank(f):
```

```

    n = len(f)
    rank = 0
    for i in range(n):
        if f[i] == 1:
            rank += 1
    return rank

n = random.randint(5, 40)
f = generate_boolean_function(n)
size_C = ac0_parity_circuit_size(f)
minRank_L = local_ring_extension_rank(f)

metric_value = minRank_L / math.log2(size_C)
conjecture_holds = minRank_L <= math.log2(size_C)
counterexample = "" if conjecture_holds else f"n={n}, minRank(L)={minRank_L}, log2(size(C))={n}

return {
    "metric_name": "Minimal Rank Ratio",
    "metric_value": metric_value,
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [2**i + 3 for i in range(5, 8)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(res["metric_value"] for res in results) / len(results)
    std_value = math.sqrt(sum((res["metric_value"] - mean_value)**2 for res in results) / len(results))
    support_fraction = sum(1 for res in results if res["conjecture_holds"]) / len(results)

    if all(res["conjecture_holds"] for res in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not res["conjecture_holds"] for res in results):
        first_failing_seed = next((seed for seed, res in zip(seeds, results) if not res["conjecture_holds"]))
        print(f"RESULT: FALSIFIED counterexample=\"n={res['instances_tested']}, minRank(L)={res['minRank_L']}\"")
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means we cannot confirm or refute the conjecture based on the results of this particular test. | next: Run the test again with increased time limits to ensure it completes and produces a result.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12519
2	preregistration	ollama_remote	glm4:latest	0	0	5904
3	novelty	ollama_remote	glm4:latest	0	0	4703
4	novelty	ollama_remote	glm4:latest	0	0	6134
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14719
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10697
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7765
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7470
9	judge	ollama_remote	glm4:latest	0	0	10744

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 80655 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/3be5f7fa76e2.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/3be5f7fa76e2.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/3be5f7fa76e2.tar.gz` (if generated)